

Procedural trees for real-time environments

Michał Piatkowski

Game Development
Bachelor
15 hp
Faculty of Technology and Society
Spring 2026
Supervisor: Georgios Palamas

Abstract—REFINE: Vegetation is a common part of procedurally generated environments. Level of detail techniques are used to reduce resource consumption based on distance without affecting perceivable quality. This thesis aims to explore applicable tree generation algorithms that could support level of detail for creating vast worlds in real-time. To this end, we present an optimized version of Space Colonization written in Rust.

Index Terms—Generative Tree Modeling, Models of Trees, Plant Growth, **TODO**:

INDEX

1. Introduction	2
1.1. Problem	2
1.2. Existing Solutions	2
1.3. Ongoing Research	2
1.4. Survey	3
2. Concepts	3
2.1. Design Scope	3
2.2. Grids	3
2.3. Voxels	3
2.3.1. Concurrent Generation	3
2.3.2. Rendering	3
2.4. Level of Detail	3
2.5. Approximation	4
2.5.1. Mimicry	4
2.6. Environmental Effects	4
3. Algorithms	4
3.1. Growing Neural Gas	4
3.2. Space Colonization	4
3.3. Neural Cellular Automata	4
4. Artifacts	4
4.1. Rendering Framework	4
4.2. Space Colonization	4
4.2.1. Data Structures	4
4.2.2. Parameters	5
4.2.3. Attractor Spawning	5
4.2.4. Versatility	5
4.2.5. Incremental Growth	5
4.3. Neural Cellular Automata	5
5. Results	5
5.1. Space Colonization	5
5.2. Neural Cellular Automata	6
6. Discussion	6
6.1. Games	6
7. Future Work	6
7.1. Scalable Noise	6
7.2. Level of Detail	6
7.3. Persistent Homology	7
Glossary	7
References	7

1. INTRODUCTION

TODO:

- these are advanced methods; mainly Neural Cellular Automata (NCA)
- sense-making: hot for modern systems and black-boxes

Procedurally generated content has seen decent traction in games over the years. Techniques come in numerous shapes and sizes, however the *survival sandbox* genre sticks out as particularly demanding across many aspects **TODO: src?**.

The initial concept for this thesis went along the lines of “procedural noise-based terrain at massive scale”, stemming from the lack of real-time procedural game worlds that utilize their endless generators for extreme view distances **TODO: src?**. Improving on this feature gap given our limited time turned out vast and vexing, even with basis in prior art **TODO: link No Man’s Sky, Minecraft, etc..** This doubt lead to an early narrowing in scope towards “procedural trees”, pivoting us into familiar yet very relevant territory.

Vegetation can make for effective decoration after all, often carrying the visuals by fleshing the world with familiar colors and improving immersion for players **TODO: src?**. Trees in particular play an important role due to their relative size in the player’s field of view, acting as subliminal landmarks [1]. We briefly continue this topic of appeal under [Section 7.3](#).

1.1. Problem

1.2. Existing Solutions

Our research puts technical aspects on the spotlight, but games are undeniably more than just engineering problems to be solved in a vacuum. Thus we leave non-technical solutions and topics up for discussion under [Section 6](#).

Since the player viewport into the virtual world is limited, we can exploit various tricks to lighten the burden on both developers and hardware. These techniques usually fall under the term *Level of Detail* (LOD); see [Section 2.4](#) for more.

TODO: billboards bad

1.3. Ongoing Research

There is as of yet no solid example of a commercial game that successfully showcases extreme view distance in a way that meaningfully integrates with gameplay **TODO: src?**. This means less representation of procedural features.

Under the space-themed part of our genre, exploration games like *Space Engineers* and *No Man’s Sky* provide long view distance almost out of necessity. Striving for realistic scales of the universe without any sort of LOD is technically infeasible **TODO: src...?**. Without a preview or map of sorts **TODO: mention Twilight Forest map**, the player would have a difficult time navigating planets, let alone choosing where to land their ship **TODO: src?**. Both games use hierarchies of voxels to store and represent terrain, albeit visualize them with *marching cubes* for a smooth look [2], compared to the blocky look people may be used to when they think of voxels **TODO: src that thought?**.

Around the *Minecraft* scene, recent technical achievements regarding LOD are modifications (mods) to the base game, made by passionate developers in the community. Working

with a large existing codebase is difficult, and this is reflected in the way LOD is approached; popular mods, such as *Distant Horizons* and *Voxy* generate simplified geometry from the ground truth, which can be framed as a mix of view-dependent and hierarchical LOD. The obvious drawback is how expensive and wasteful this process can be, generating fully detailed chunks when only a few upper chunk sections happen to be visible.

<https://github.com/xandergos/terrain-diffusion-mc>

TODO: mention MC terrain diffusion mod?

TODO: for Section 6: theory on how noise-based terrain could be inherently generated at different scales

TODO: mention MC sections and LOD mods

TODO: mention <https://veloren.net/>, apparently

TODO: mention ray/path-tracing and BVH acceleration? especially Nanite from UE5, and voxel-Lumen for foliage

1.4. Survey

Given the state of things, we can finally revisit our problem; why do procedurally generated games struggle to present the vastness of their worlds? This turns out to be a vast question in its own right **TODO: intertwine or merge with intro motivation?**. A simple theory is that long view distances need too large of a technical investment for an unproven feature **TODO: sources?!**.

TODO: what our research adds to this area (algos?)

2. CONCEPTS

2.1. Design Scope

When crafting worlds for games, the designer tends to distribute resources and skills in a way that ends up noticeable to the player **TODO: src?**. This tendency has a decisive effect in the context of visual features; nearby objects get more attention from the player and in turn the designer, while distant objects lose importance. In linear or closed-world games, the designer can deliberately choose where to allocate resources since playable area is limited **TODO: src?**. Open-world games, especially those relying on procedural generation, greatly complicate such decisions. We can no longer focus on discrete parts but have to instead produce a coherent whole. To cope with the vast search space, the designer may intentionally impose rules and constraints, forming a *design language* with basis in available resources, target audience, and the artistic vision **TODO: src?**.

2.2. Grids

Partitioning space into grids can be implicit like rooms in a dungeon, or explicit like individual voxels in terrain. Herein lies the tradeoff; grid size becomes one of the deciding factors on how the designer spends their resources.

Take some of the possible features that can appear in an infinite sandbox game like terrain, biomes, vegetation, or structures **TODO: src?** – listed roughly from large to small. These need to be mixed in some way to make an engaging experience, but in a procedural world, we don't know where the player will venture or the order in which they encounter features. We can guide them with specific cues, but it can be

challenging to generate meaningful trails, such as dirt paths leading to settlements, or landmarks that signal some nearby point of interest.

Hardware limitations are a driving force as well, where grids can be particularly versatile in accommodating memory limits, but also introduce parallelization opportunities.

TODO: section feels incomplete

2.3. Voxels

Partitioning space can help us with *separation of concerns* in the distributed system that is our game world **TODO: src?**. To better explain how one can utilize a hierarchy of grids, we can use *Minecraft* as an example:

- 1) *Blocks* are the smallest unit i.e. the voxels.
- 2) *Sections* are portions containing 16^3 blocks.
- 3) *Chunks* are columns of sections – height can vary.
- 4) *Regions* are groups of 32×32 chunks.

Anything beyond blocks is practically just technical details for a player, but to us it can explain some technical intricacies.

2.3.1. Concurrent Generation: The world is generated chunk-wise and over multiple consecutive steps, all on demand as chunks appear in the player's view distance [3]. Structure origins and biomes are calculated before blocks can be placed. Terrain shaping follows, and is a computationally expensive phase thanks to various density functions. The convenient property of these formulas is their continuity, given that most of them are multiple octaves of gradient noise with some linear arithmetic sprinkled in between. Density calculations are deterministic and embarrassingly parallel, in other words.

Caves and cliffs are fun and all, but there is a concurrency problem soon after; the decoration post-processing steps that may place structures, vegetation, and ore – to name a few. The point of congestion is a possibility of decorations intersecting multiple chunks, which is solved by tracking such dependencies and finalizing later.

TODO: section feels somewhat incomplete

2.3.2. Rendering: Chunks render only when fully generated, which is normal across games and avoids the overhead of rapidly shifting previews.

TODO: mention rendering, and transition into next section

2.4. Level of Detail

At the beginning of 3D graphics, avoiding unnecessary work was crucial to render scenes in real-time. Rasterization was the bottleneck, so painting only visible parts of polygons front-to-back could skip a lot of processing [4]. The bottleneck has since shifted wildly, orienting the solution space around software design rather than hardware limitations, yet the spirit of avoiding work lives on. This leads us to a perhaps literal interpretation of approximation – LOD – referring to the complexity of a 3D model, commonly recognized by the act of reducing detail based on distance [5]. Triangle meshes can be simplified by removing vertices that contribute little based on perspective, most often baked algorithmically into a couple variants as part of an offline content pipeline. At the extreme end, a 3D model is pre-rendered into a 2D sprite, known as a *billboard*.

2.5. Approximation

This premise of estimates applies across disciplines and is an essential tactic for simulating complex natural phenomena on constrained digital hardware **TODO: src?**.

TODO: feels incomplete

2.5.1. *Mimicry*: Borrowing ideas from nature may be one of the most self-evident approaches in replicating nature-like structures and behaviors. More importantly, we do not need to simulate life in microscopic detail to recreate useful mechanisms, as will become evident in [Section 3.2](#). For games we are usually encouraged to bend the rules, especially if the artistic vision is not in need of realism, as objects generally have to fit the motive of a given media to not feel out of place **TODO: src?**.

2.6. Environmental Effects

TODO: Explain L-systems mentioned in [6]

TODO: Explain realistic tree models found in [7] and [8]

TODO: Explain how environment/weather/light affects tree growth [9]

3. ALGORITHMS

Methods listed in this section follow the idea outlined by [Section 2.5](#).

TODO: intro paragraph:

- real-time generation for games

3.1. Growing Neural Gas

REFINE: Supervisor suggested checking out Growing Neural Gas (GNG) networks [\[10\]](#) when the initial goal was established. Even after exploring various GNG implementations, we concluded that GNG was not particularly relevant to the subject of growing trees, nor was it effective for real-time even with major optimizations [\[11\]](#).

Possible uses we came up with were related to creating bounding geometry for tree crowns. These could be used to spawn Space Colonization (SC) attractors (more on that later), or to construct meshes for rendering (which grew out of scope).

On the topic of GNG, we also explored the LOD aspect and found interesting applications for compressing point cloud colors [\[12\]](#), but without relevance to *growing* interesting trees.

3.2. Space Colonization

TODO: we can classify the methods, starting with SC: this is a space-filling algo, what else exists?

TODO: The authors of [13] are proud that their solution “grows like actual trees”. Ties to [Section 2.5](#).

TODO:

- Algo and how it’s good for trees [\[13\]](#)
- Shadow propagation based approach mentioned in [\[8\]](#)
- How environment/weather/light can be used to affect SC [\[9\]](#)

TODO:

- mention R* tree [\[14\]](#) (found by examining [\[15\]](#)) as critical optimization over other data-structures
- evaluate performance claims in [\[16\]](#)

3.3. Neural Cellular Automata

TODO: Explain the algo and how it may be an interesting avenue for vegetation [17]

TODO: Mention other attempts and uses outside of trees [18], [19]

4. ARTIFACTS

4.1. Rendering Framework

To ease experiments regarding our algorithms of choice, we develop a minimal framework with the primary purpose of visualizing point clouds, since most of the data we are working with is effectively represented by points.

The choice of tech stack was driven by the performance-oriented nature of this research, which is why we settled on Rust early and out of familiarity. This gave us access to high-quality libraries that provide essentials like graphics API (`wgpu`), immediate mode GUI (`egui`), and acceleration structures (`rstar`).

TODO: Write more?

4.2. Space Colonization

We implemented the SC algorithm from the ground up by following the method section for tree modeling [\[13\]](#). Nearest Neighbor Search (NNS) is a fundamental part of the algorithm, and a $O(n^2)$ loop over all attractors is untenable.

4.2.1. *Data Structures*: Thus an octree library (`oktree`) was used as the first acceleration structure, getting us close to real-time generation. Octrees served us well throughout development, but it became apparent that iterating over many large overlapping spheres was expensive when influence radii d_i on attractors was large. Naively constraining the search to a certain radius would deviate us from the algorithm, in turn worsening growth dynamics by disregarding distant attractors.

Realising that we worked with volumes and not just points, we pursued a spatial hash with cells that each referenced all intersecting attractors. These amortized lookups gave a decent speedup, but increased both memory usage and removal time due to excessive duplication. Cost remained high even while utilizing packed references as small as 16-bit; enough for 65535 attractors, or 2^{16} indices minus 1 tombstone. Commonly referred to as a *memory arena*, it allows self-referential indices and reduces data movement **TODO: src?**.

The structures we explored usually trade construction time and memory for improved lookup times. This was the reason for us looking beyond Voronoi diagrams in the first place, specifically with the expensive Delaunay triangulation step needed to construct them. An aspect we paid less attention to, but is worth noting, is tree balance and quality, which can vary at runtime as attractors are killed and nodes are spawned.

So far, neither octrees or spatial hashing proved effective against slowdown caused by smaller segment size D . Our supervisor seemed to recognize this problem as related to DBSCAN [\[15\]](#), leading to our final structure of choice; the R*-tree. Using an existing library (`rstar`), we managed to further decrease lookup time and trivialize construction cost while creating an optimal tree since we have all points upfront,

all while maintaining fast NNS regardless of influence radius. This was now a faithful reimplementation of the SC algorithm.

5. RESULTS

5.1. Space Colonization

4.2.2. *Parameters:* **TODO:** explain how parameters can be expanded upon, e.g. alternating node activations, non-immediate attractor death

The main variables d_i , d_k , and D can lead to very different generation times for a particular attractor cloud. Smaller segments lead to fewer attractors potentially being killed per new node, increasing the amount of iterations needed to consume all reachable attractors. Our choice of data structures may greatly alleviate the cost of large influence distances, but this variability can fundamentally not be eliminated without a different growth strategy and would likely require a new approach overall.

4.2.3. *Attractor Spawning:* **TODO:** describe how the cloud shapes the tree

TODO: Using Signed Distance Field (SDF) to spawn attractors

4.2.4. *Versatility:* With our definition of real-time **TODO:** define real-time in intro maybe?, we luckily do not need to worry about achieving predictable time complexities. We can tune both parameters and data, allowing designers to also allocate time towards post-processing of the generated skeleton. Point clouds are unlikely to be useful in their raw form, **TODO:** incorporate [7] ref but the methods for sprucing them up are vast.

The SC paper already mentions decimation and subdivision along branch curves, but these operate on points to improve detail. An example that is more relevant to us, and closer to the tail end of the content pipeline, is voxelization; the act of turning smooth geometry into a rigid grid **TODO:** any good reference?. **TODO:** shortly describe point->voxel process (and e.g. ->MC block)

4.2.5. *Incremental Growth:* **TODO:** SC can be grown over multiple steps (or over multiple game frames) allowing for massive structures without a hitch

4.3. Neural Cellular Automata

TODO: Describe the experiment around NCA [18]:

- Reproducing the open-source build found on GitHub by updating Python packages.
- Exploring the provided Jupyter notebook.
- Modifying the loss function to achieve different growth patterns. The original loss function is a combination of *Softmax* and *negative log likelihood* loss¹.

TODO:

- black box / related to evolutionary algo
- difficult for algo to settle on novel shapes

TODO:

- pruning methods; why is it important, how to measure?
- how easy is it to control appearance?
- run a test? benchmark?
- would like to see: insights around NCA loss function (helps reach a goal), not particularly a ready product

TODO: SC reimplementation:

- We tested custom spatial hash buckets and octrees (slowest construction, unbalanced storage, unbalanced lookup time).

Elaborate on

- default vs. tuned insertion parameters for R* tree
- customized spatial hashing

¹<https://docs.pytorch.org/docs/stable/nn>

TODO: Improve readability of these images...
and add more interesting images (probably as appendix)

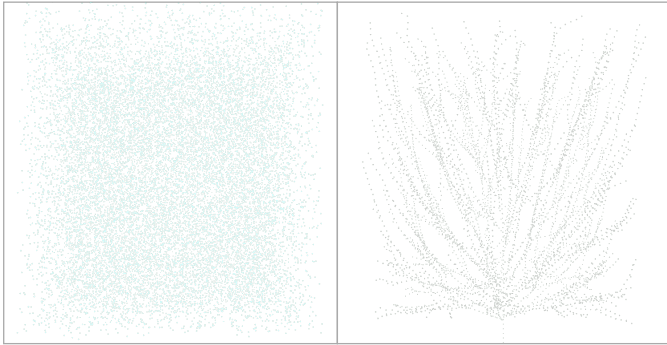


Figure 1. SC box before growth.

Figure 2. SC box after growth.

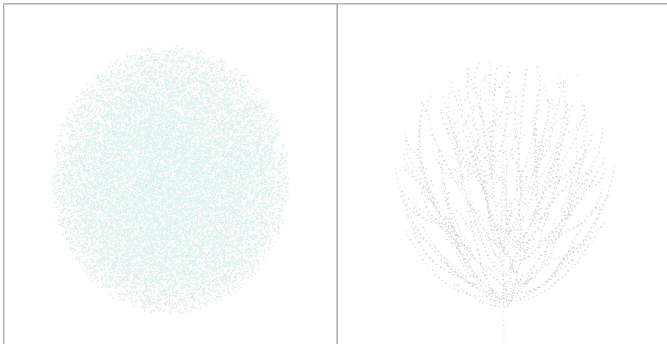


Figure 3. SC egg before growth.

Figure 4. SC egg after growth.

TODO:

- demonstrate parametrized examples from exploration
- variation comes from a combination of point cloud shape, influence distance, kill distance.

TODO: Write about data structure exploration and explain Table 1.

TODO: sift through git and get exact timings for buckets and octree, and maybe be more concise about the Memory column...

	Construct	Lookup	Memory
Octree	100ms	1500ms	Low
Spatial Hash	200ms	800ms	High
R*-tree	1.5ms	300ms	Low

Table 1. Data structure metrics for 20000 randomly distributed attractors.

TODO: Hardware:

- CPU: AMD Ryzen 7 7700X 8-Core Processor
- RAM: 2 × 32 GB, 5200 MT/s, CL32

5.2. Neural Cellular Automata

REFINE: Preliminary result around NCA [18]:

- Growth quickly collapses by scaling weights with values below one.
- Introducing more randomness to cell propagation creates spurious growths, which may be effective for tree variation, but may appear too close to unnatural overgrowth without introducing new restrictions.



Figure 5. Original NCA tree

Figure 6. Experimental NCA tree with factors: mask = 0.2, fire = 0.75

6. DISCUSSION

TODO:

- Compare our performance with original SC Voronoi structure (slowest construction, balanced storage, balanced lookup time).
- Compare our customizability with mentioned papers regarding tree modeling.

TODO: Describe how thesis progressed from a goal (generating trees in real-time for games) to exploring various prior work ([10], [13], [17])

TODO:

- What methods can be combined, and how?
- Loop back to LOD and reason around feasibility.

TODO:

- Compare SC vs shadow propagation [8]
- Light and shadow are key to creating nature-like trees.

TODO: Discuss future work from SC papers: convert scanned point clouds of real trees to algo params

TODO: Explain relevance of environment in tree growth:

- Local species (pests, competition, mycelium) may act as biomes to dictate tree attributes.

Generating a population of trees:

- quick poisson disc sample for initial placement?
- elaborate on how terrain can shape placement

6.1. Games

Terraria: **TODO:** Mention Terraria “Living Trees”? They were important inspiration after all...

Minecraft: **TODO:** Mention Minecraft mods that were used in thesis slides? Sounds fair to mention, considering that we can look at their code and see how large trees/structures were generated in practice. This gives us a good glimpse into upsides and pitfalls.

7. FUTURE WORK

7.1. Scalable Noise

TODO: expand on Section 1.2; “generating noise at different LOD”

7.2. Level of Detail

TODO: few insights into potential level-of-detail support (which was the initial motivation/goal).

GLOSSARY

API	Application Programming Interface
GNG	Growing Neural Gas
GUI	Graphical User Interface
LOD	Level of Detail
NCA	Neural Cellular Automata
NNS	Nearest Neighbor Search
SC	Space Colonization
SDF	Signed Distance Field

INDEX OF FIGURES

Fig. 1 SC box before growth.	6
Fig. 2 SC box after growth.	6
Fig. 3 SC egg before growth.	6
Fig. 4 SC egg after growth.	6
Fig. 5 Original NCA tree	6
Fig. 6 Experimental NCA tree with factors: mask = 0.2, fire = 0.75	6

INDEX OF TABLES

Table 1 Data structure metrics for 20000 randomly distributed attractors.	6
---	---

REFERENCES

- [1] P. H. Jøger, N. C. Nilsson, and G. Palamas, "Can't see the Forest for the Trees: Perceiving Realism of Procedural Generated Trees in First-Person Games," *EAI Endorsed Transactions on Creative Technologies*, vol. 8, no. 26, 2021, doi: [10.4108/eai.16-3-2021.169029](https://doi.org/10.4108/eai.16-3-2021.169029).
- [2] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," *SIGGRAPH Comput. Graph.*, vol. 21, no. 4, pp. 163–169, Aug. 1987, doi: [10.1145/37402.37422](https://doi.org/10.1145/37402.37422).
- [3] "Minecraft Wiki." Accessed: May 20, 2026. [Online]. Available: <https://minecraft.wiki/>
- [4] D. Gordon and S. Chen, "Front-to-back display of BSP trees," *IEEE Computer Graphics and Applications*, vol. 11, no. 5, pp. 79–85, 1991, doi: [10.1109/38.90569](https://doi.org/10.1109/38.90569).
- [5] T. K. Heok and D. Daman, "A review on level of detail," in *Proceedings. International Conference on Computer Graphics, Imaging and Visualization, 2004. CGIV 2004.*, 2004, pp. 70–75. doi: [10.1109/CGIV.2004.1323963](https://doi.org/10.1109/CGIV.2004.1323963).
- [6] O. Blomqvist, P. Kraft, H. Lidin, R. Motzheim, A. Tonderski, and G. Wagner, "Generating Compelling Procedural 3D Environments and Landscapes," 2016. [Online]. Available: <https://hdl.handle.net/20.500.12380/244588>
- [7] R. Hoetzlein, "A Procedural Model for Diverse Tree Species," in *Proceedings of the 17th International Conference on the Foundations of Digital Games*, in FDG '22. Athens, Greece: Association for Computing Machinery, 2022. doi: [10.1145/3555858.3564251](https://doi.org/10.1145/3555858.3564251).
- [8] W. Palubicki *et al.*, "Self-organizing tree models for image synthesis," in *ACM SIGGRAPH 2009 Papers*, in SIGGRAPH '09. New Orleans, Louisiana: Association for Computing Machinery, 2009. doi: [10.1145/1576246.1531364](https://doi.org/10.1145/1576246.1531364).
- [9] S. Pirk, T. Niese, T. Hädrich, B. Benes, and O. Deussen, "Windy trees: computing stress response for developmental tree models," *ACM Trans. Graph.*, vol. 33, no. 6, Nov. 2014, doi: [10.1145/2661229.2661252](https://doi.org/10.1145/2661229.2661252).

Future: properly investigate the viability of these algorithms in a real-time environment, with the crux being that it should be scalable for large procedural worlds.

TODO: minimal rendering framework; the trees are not interesting/novel compared to prior work, and visualization of experiments was hindered by the framework.

Future: better rendering to

- 1) spend less time in debugging/testing
- 2) attempt reaching parity with [8] in particular

7.3. Persistent Homology

TODO: Measure/identify tree shape with "barcodes" [20]

TODO: <https://github.com/peterbraden/genetic-lisa>

- related to [20] and
- abstract triangles vs rigid cells (of NCA)
- genetic/neural algo to spawn SC attractors?

TODO: We may exploit this through distinct mutations, with one of the most obvious variations being tree height when accounting for distance **TODO:** src ...me? find science?.

TODO: tie back with [1]

- [10] B. Fritzke, “A growing neural gas network learns topologies,” *Advances in neural information processing systems*, vol. 7, 1994.
- [11] D. Fišer, J. Faigl, and M. Kulich, “Growing neural gas efficiently,” *Neurocomputing*, vol. 104, pp. 72–82, 2013, doi: [10.1016/j.neucom.2012.10.004](https://doi.org/10.1016/j.neucom.2012.10.004).
- [12] S. Orts-Escolano, J. Garcia-Rodriguez, V. Moreli, M. Cazorla, and J. M. Garcia-Chamizo, “3D colour object reconstruction based on Growing Neural Gas,” in *2014 International Joint Conference on Neural Networks (IJCNN)*, July 2014, pp. 1474–1481. doi: [10.1109/IJCNN.2014.6889546](https://doi.org/10.1109/IJCNN.2014.6889546).
- [13] A. Runions, B. Lane, and P. Prusinkiewicz, “Modeling trees with a space colonization algorithm,” in *Proceedings of the Third Eurographics Conference on Natural Phenomena*, in NPH’07. Prague, Czech Republic: Eurographics Association, 2007, pp. 63–70. doi: [10.5555/2381384.2381395](https://doi.org/10.5555/2381384.2381395).
- [14] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, “The R*-tree: an efficient and robust access method for points and rectangles,” in *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*, in SIGMOD ’90. Atlantic City, New Jersey, USA: Association for Computing Machinery, 1990, pp. 322–331. doi: [10.1145/93597.98741](https://doi.org/10.1145/93597.98741).
- [15] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, in KDD’96. Portland, Oregon: AAAI Press, 1996, pp. 226–231. doi: [10.5555/3001460.3001507](https://doi.org/10.5555/3001460.3001507).
- [16] Y. Narasimhulu, V. Venkaiah, R. Pasunuri, and A. Suthar, “CKD-Tree: An Improved KD-Tree Construction Algorithm,” in *ISIC 2021: International Semantic Intelligence Conference*, New Delhi, India, 2021, pp. 211–218. [Online]. Available: <https://ceur-ws.org/Vol-2786/Paper28.pdf>
- [17] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, “Growing Neural Cellular Automata,” *Distill*, 2020, doi: [10.23915/distill.00023](https://doi.org/10.23915/distill.00023).
- [18] S. Sudhakaran *et al.*, “Growing 3D Artefacts and Functional Machines with Neural Cellular Automata,” 2021. doi: [10.1162/isal_a_00451](https://doi.org/10.1162/isal_a_00451).
- [19] D. Zhang, C. Choi, J. Kim, and Y. M. Kim, “Learning to Generate 3D Shapes with Generative Cellular Automata,” 2021. doi: [10.48550/arXiv.2103.04130](https://doi.org/10.48550/arXiv.2103.04130).
- [20] A. J. Kemme and C. A. Agyingi, “Persistent Homology: A Pedagogical Introduction with Biological Applications,” 2025. doi: [10.48550/arXiv.2505.06583](https://doi.org/10.48550/arXiv.2505.06583).